FITBUDDY

AI Fitness Plan Generator using Gemini Models

PROJECT REPORT

A web-based AI application for personalized workout plans, nutrition/recovery tips, feedback-based plan updates, and administrative monitoring.

Technology Stack

FastAPI • Google Gemini • SQLAlchemy • SQLite • Jinja2 • HTML/CSS • Uvicorn

Project Description:

FitBuddy is a web-based application that uses AI to generate personalized workout plans and nutrition tips based on a user’s fitness goals, such as weight loss, muscle gain, or general wellness. Built with FastAPI and integrated with Google’s Gemini AI models, FitBuddy simplifies the process of creating structured and effective fitness routines.

The application takes in user information—like age, weight, goal, and preferred workout intensity—and generates a customized 7-day fitness plan. Users can also submit feedback to update and refine their plan using AI. A complementary nutrition or recovery tip is also provided based on their selected goal.

With a clean user interface and database support via SQLite, FitBuddy ensures each user's data and generated plan are stored and accessible. It offers both API endpoints and an interactive HTML interface for ease of use.

Scenarios:

Scenario 1 : A user inputs their personal details including name, age, weight, fitness goal (e.g., “weight loss” or “muscle gain”), and preferred workout intensity (“high”, “medium” or “low”) to receive a personalized 7-day workout plan. The system generates a day-by-day schedule tailored to the user's fitness objective and experience level. This AI-driven customization enhances user motivation and adherence by providing structured and goal-specific routines.

Scenario 2: A user submits feedback on their existing workout plan, such as suggesting changes like “more focus on cardio” or “include more rest days.” The system uses this input to regenerate an updated workout plan, refining it based on the user's evolving needs. This feedback loop ensures that users stay engaged and receive dynamic fitness support as their preferences or progress change.

Scenario 3: A user requests a nutrition or recovery tip based on their selected fitness goal. The platform delivers a concise and relevant suggestion (e.g., “Include protein in your post-workout meal” for muscle gain), which complements the workout plan. These targeted tips help users improve results by aligning their diet and recovery practices with their physical training regimen.

Scenario 4: An admin or coach accesses the platform to view all registered users and their workout plans through a dedicated dashboard. This view allows them to track individual progress, compare original and updated plans, and monitor user activity. It streamlines oversight for trainers managing multiple clients or students.

Technical Architecture:

Pre - requisites:

FastAPI Framework Knowledge: FastAPI Documentation

Gemini API Familiarity: Google Generative AI Documentation

HTML, CSS, and Jinja2 Template Skills: W3Schools HTML/CSS/Jinja2 Tutorials

Python Programming Proficiency: Python Official Docs

Version Control with Git: Git Documentation

SQLAlchemy and SQLite Basics: : SQLAlchemy Docs

Environment Setup (pip, virtualenv, or conda): Virtualenv Guide

Uvicorn ASGI Server: Uvicorn Docs

Project Workflow:

Milestone 1. Model selection and Architecture

Activity 1.1. Research and Select the Appropriate Generative AI Model

Activity 1.2. Define the architecture of the application

Activity 1.3. Set up the development Environment

Milestone 2. Core functionalities Development

Activity 2.1. Develop the core functionalities

Activity 2.2. Implement the FastAPI Backend to Manage Routing and User Input Processing

Milestone 3. route.py Development

Activity 3.1. Writing the Main Application Logic in routes.py

Milestone 4. Frontend Development

Activity 4.1. Designing and Developing User Interface

Activity 4.2. Creating Dynamic Templates with FastAPI’s Jinja2

Milestone 5. Deployment

Activity 5.1. Preparing the Application for Local Deployment

Activity 5.2. Testing and Verifying Local Deployment

Milestone 1. Model selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI models from Google’s Gemini family for the FitBuddy application. This includes evaluating the performance and capabilities of multiple Gemini models to ensure they align with the application’s core functionalities—such as generating personalized workout plans, updating them based on feedback, and providing tailored

nutrition or recovery advice.

Activity 1.1: Research and Select the Appropriate Generative AI Model

Understand the Project Requirements

FitBuddy’s AI model needed to:

Generate structured 7-day workout plans

Provide concise and actionable nutrition tips

Update workout plans based on user feedback

Perform well in real-time, web-based interaction

Model Evaluation

We initially experimented with several Hugging Face models:

T5 & BART – For text generation (workout/nutrition)

DistilBERT – For potential intent classification

Bloom – For general purpose generation

While these models showed promise in basic text generation, they had notable limitations.

These models required:

Hosting with Hugging Face or local GPU setup

Fine-tuning or complex prompting to achieve structured plans

Final Model Selection

After comparative testing, we selected Google Gemini models for production:

Gemini 1.5 Pro → Used for structured 7-day workout generation and plan updating → Delivers rich, context-aware, formatted responses reliably

Gemini Flash → Used for generating fast, practical nutrition tips → Lightweight and efficient for quick API calls

Reasons for Choosing Gemini:

API-based, easy to integrate with FastAPI

Naturally understands structured formatting (days, sections)

High accuracy with minimal prompt tuning

Scalable and fast for feedback-based revisions

Activity 1.2.Define the architecture of the application

Architecture Overview: FitBuddy follows a modular FastAPI-based architecture-

Frontend: HTML + Jinja2 templates for user input and display

Backend: FastAPI for route handling and logic

AI Layer: Google Gemini APIs (Pro & Flash)

Database: SQLite with SQLAlchemy for persistence

Frontend Responsibilities: Users interact via-

index.html – Input form

result.html – Plan, tip, feedback

all_users.html – Admin panel

Backend Responsibilities:

Receives form data

Calls Gemini models

Stores user info and plans

Renders dynamic templates

AI Integration Points:

Workout generation & feedback updates → Gemini Pro

Nutrition tips → Gemini Flash

Activity 1.3 Set up the development Environment:

Install Python & Pip: Ensure Python and pip are installed.

Create Virtual Environment: python -m venv fitbuddy-env fitbuddy-env\Scripts\activate (Windows)

Install Required Libraries: pip install fastapi uvicorn jinja2 sqlalchemy python-multipart google-generativeai

```
Run Server: uvicorn app.main:app --reload Visit http://127.0.0.1:8000 for the app and /docs for API testing.
```

Set Up Project Structure: Create folders: /app (Python files), /templates (HTML), /static/images (assets)

Milestone 2. Core functionalities Development

Activity 2.1 Develop the core functionalities:

Workout Plan Generation

Function: generate_workout_gemini() File: gemini_generator.py

Implements the core logic to generate a personalized 7-day workout plan using user inputs such as goal (e.g., muscle gain, weight loss) and intensity (low, medium, high). The prompt is sent to the Gemini 1.5 Pro model, which returns a structured, day-wise plan including:

Warm-up (5–10 mins)

Main workout (exercise details, sets, reps)

Cooldown or recovery tip

The output is returned to routes.py and rendered on the result.html page using <pre> blocks for formatting.

Nutrition Tip Generation

Function: generate_nutrition_tip_with_flash() File: gemini_flash_generator.py

This function generates a concise, practical nutrition or recovery tip tailored to the user’s selected goal. It utilizes the Gemini Flash model to deliver a fast, helpful response such as hydration advice, protein suggestions, or general recovery best practices. This tip is displayed beneath the workout plan in result.html.

Feedback-Based Plan Updating

Function: update_workout_plan() File: updated_plan.py

This function enables the user to revise their original plan by submitting feedback (e.g., "Add yoga", "Include more cardio"). The original plan and feedback are passed to Gemini 1.5 Pro, which returns a modified plan with the suggested updates. The updated plan is then stored and displayed alongside the new nutrition tip.

User & Plan Storage

Functions:

save_user(), save_plan(), update_plan()

get_original_plan(), get_user()

File: database.py

All user information (ID, name, age, weight, goal, intensity) and AI-generated plans are stored in a SQLite database via SQLAlchemy ORM. Feedback-based updates are saved as a separate field to preserve both versions (original and updated).

Admin View of All Users

Route: /view-all-users Template: all_users.html

Displays a list of all users and their respective plans. Rendered with Jinja2 using a {% for user in users %} loop, it allows administrators to see original and updated plans clearly in a structured table format.

Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing

Define Routes in FastAPI

All routing logic is handled in routes.py. Each route is linked to the corresponding function developed in Activity 2.1.

Process User Input

HTML Form: Created in index.html

Users enter: username, user_id, age, weight, goal, and intensity

FastAPI captures inputs using Form(...) parameters

Pydantic schemas (UserInput, FeedbackRequest) are used to validate structure

Integrate Gemini API Calls

Each route handler connects directly to the respective Gemini-powered function:

/generate-workout →

generate_workout_gemini() for workout plan

generate_nutrition_tip_with_flash() for tip

/submit-feedback →

Retrieves plan

Sends original + feedback to update_workout_plan()

All AI-generated responses are returned cleanly and injected into the frontend templates for display.

Milestone 3. routes.py Development

Milestone 3 focuses on building the routing logic inside the routes.py file, which acts as the core operational layer of the FitBuddy application. It bridges the frontend templates, AI generation modules (Gemini Pro and Flash), and the SQLite database. Each route defines how user inputs are collected, processed, and responded to — creating a dynamic, personalized experience for fitness planning and feedback-based updates.

Activity 3.1: Writing the Main Application Logic in routes.py

Defining Core routes in routes.py:

Separate routes were implemented to serve each of FitBuddy's main functionalities:

/ : Displays the user form via index.html

/generate-workout: Processes user input and generates a personalized workout plan

/submit-feedback: Updates the workout plan based on user feedback

/view-all-users: Displays an admin dashboard showing all users and their plans

Set up Route Handlers for Each Feature

- / – Home Route

Returns the homepage (index.html)

No form processing logic

- /generate-workout – Plan Generator

Receives user input using FastAPI's Form(...)

Constructs a UserInput Pydantic model

Calls:

generate_workout_gemini(...) → Generates workout plan via Gemini 1.5 Pro

generate_nutrition_tip_with_flash(...) → Gets nutrition tip via Gemini Flash

Stores:

User details with save_user(...)

Plan with save_plan(...)

Returns result.html with dynamic data

- /submit-feedback – Update Plan with Feedback

Captures feedback and user_id

Retrieves original plan from DB

Uses update_workout_plan(...) to revise the plan via Gemini 1.5 Pro

Updates the revised plan in the DB using update_plan(...)

Renders updated result on result.html

- /view-all-users – Admin View

Calls get_all_users() and get_all_plans() to fetch DB records

Passes them to all_users.html

Admin can view:

Username, Age, Weight, Goal, Intensity

Original and updated workout plans

Integrate Gemini AI Responses in Each Function

Each AI-related route calls Google’s Gemini models with specific prompts:

Workout Plan Generation: generate_workout_gemini() sends fitness goal and intensity to Gemini Pro to return a structured 7-day workout plan.

Nutrition Tip Generation: generate_nutrition_tip_with_flash() uses Gemini Flash to generate a practical tip related to the user’s fitness goal.

Workout Plan Update via Feedback: update_workout_plan() passes original plan + user feedback to Gemini Pro and returns a revised plan.

All responses are returned to the user through result.html, maintaining clarity using <pre> blocks for formatting.

Workout generation route Nutrition/Recovery tip generation route

Nutrition/Recovery tip generation route

Saving user info and plan generation route

Submit Feedback and update plan route

Milestone 4. Frontend Development

Milestone 4 focuses on developing a user-friendly and visually structured interface for FitBuddy, the AI-powered workout generator. This involves building responsive and interactive web pages using HTML, CSS, and Jinja2 templating. Each template is dynamically rendered through FastAPI, ensuring users can easily submit input, receive personalized plans, provide feedback, and allow administrators to manage data effectively. This milestone guarantees an engaging and intuitive fitness planning experience for all users.

Activity 4.1 Designing and Developing the User Interface

Set Up the Base HTML Structure

Developed index.html as the main entry point for user input.

Structured the form to capture:

username, user_id, age, weight, goal, and intensity.

Each field is properly labeled and grouped using semantic HTML.

The design uses a gym-themed background image, Google Fonts (Roboto), and bold typography to create a clean, fitness-oriented interface.

Navigation is kept minimal for focus, and user flow directs clearly from input to output.

Design a Responsive Layout Using CSS

Embedded CSS styles were applied directly in each HTML file.

Layouts use Flexbox to center content both vertically and horizontally.

Media queries ensure mobile responsiveness.

Buttons have consistent styling with hover effects.

Input fields and result blocks have adequate padding, shadows, and rounded corners for modern aesthetics.

Overall color scheme supports readability on a dark background.

Create Separate Pages for Each Core Functionality

Three Jinja2-powered HTML templates were created under /templates:

index.html: A user input form with fields for fitness goal, intensity, and basic personal information.

result.html: Displays the 7-day AI generated workout plan, a nutrition tip, and accepts user feedback.

all_users.html: Admin panel to view and delete users and review both original and updated plans.

Templates are modular, and UI transitions smoothly between steps.

Forms are submitted to specific FastAPI endpoints via POST methods

Activity 4.2 Creating Dynamic Templates with FastAPI's Jinja2

Integrate Jinja2 Templating for Dynamic Content Rendering

Used FastAPI's Jinja2Templates for template rendering:

```
from fastapi.templating import Jinja2Templates

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

TEMPLATE_DIR = os.path.join(BASE_DIR, "templates")

templates = Jinja2Templates(directory=TEMPLATE_DIR)
```

Each route uses TemplateResponse to render the proper HTML with passed context:

This allows dynamic plan and nutrition tip display after form submission.

Bind Backend Data to HTML Templates

- index.html

Form elements use name="..." to bind to backend Form(...) parameters.

Submits data to /generate-workout.

- result.html

Uses:

{{ username }}, {{ goal }}, {{ intensity }}, etc. to show user data.

{{ workout_plan }} and {{ nutrition_tip }} to show AI-generated results.

Includes a feedback form submitted via POST to /submit-feedback.

- all_users.html

Loops over all user records using:

Milestone 5. Deployment

Now, the focus is on deploying the FitBuddy application on a local system using FastAPI and Uvicorn. The app uses HTML templates rendered with Jinja2, alongside AI services powered by Gemini. This milestone ensures all environment dependencies are correctly installed, and the app runs smoothly on a local server, preparing it for future deployment on cloud platforms.

Activity 5.1: Preparing the Application for Local Deployment

Set up a virtual environment

Create a virtual environment to manage dependencies and isolate the FitBuddy project from other environments.

Activate the environment and install all necessary dependencies listed in the requirements.txt file, including FastAPI, Uvicorn, SQLAlchemy, and Google Generative AI SDK.

```
python -m venv venv

source venv/bin/activate # On Windows: venv\Scripts\activate

pip install -r requirements.txt
```

Configure environment variables

Use environment variables to store sensitive data such as the Gemini API Key.

These variables can be set directly in the terminal or added to a .env file which is loaded into the application using python-dotenv.

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

Frontend-Backend Rendering via Jinja2

All user interactions are facilitated via Jinja2-rendered templates:

index.html: Collects user inputs

result.html: Displays the generated AI plan

all_users.html: Lists all user records for admin view

Activity 5.2: Testing and Verifying Local Deployment

Access the Application Locally

```
The FastAPI development server must be started using the command (uvicorn main:app
–reload). Once the server is running, the application can be accessed through any web
browser by navigating to http://127.0.0.1:8000. This URL loads the main user interface
where individuals can input their details and generate personalized fitness plans.
Additionally, the interactive API documentation provided by FastAPI can be viewed at
http://127.0.0.1:8000/docs, allowing developers to explore and test available endpoints.
```

Exploring Website’s web pages:

Home page:

Description: The homepage of FitBuddy – AI Workout Generator features a clean and engaging user interface designed to provide a seamless experience for users seeking personalized fitness plans. The form prominently displayed in the center allows users to enter essential information including:

Name

User ID

Age

Weight (kg)

Fitness Goal (e.g., weight loss, flexibility)

Workout Intensity (Low, Medium, High)

Once users submit this data, they can click the "Generate Plan" button, which triggers the AI-powered backend to generate a customized workout and nutrition plan tailored to their goals and fitness profile. The visual appeal of the background image featuring an active individual reinforces the fitness-centric theme of the application, motivating users to take charge of their health journey.

This intuitive design ensures users of all experience levels can quickly get started with minimal effort.

Personalized workout generation page:

User Information:

Description: Once the user submits their details, the application navigates to a personalized workout plan page. This section displays a summary of the user's input including their Name, User ID, Age, Weight, Fitness Goal, and Selected Intensity. This personalized display ensures users feel acknowledged and guides the AI engine to generate relevant workout recommendations tailored specifically to their fitness journey.

Workout Plan:

Description: FitBuddy generates a personalized, day-wise workout plan tailored to the user's fitness goal, experience level, and preferred workout schedule. The workout section on the result page outlines each day with a clear focus, such as full body, upper or lower body, cardio, core strength, or flexibility. For every workout day, a curated list of exercises is provided, detailing the name of the exercise, the number of sets and repetitions (or duration), and suggested rest intervals. The plan incorporates variety and structured progression to keep the workouts engaging and effective. Additionally, basic warm-up and cool-down guidance may be included to support safe and well-rounded training. Overall, this section aims to deliver a goal-oriented, easy-to-follow workout schedule that promotes consistency and progress.

Nutrition Tip:

Description: The Nutrition Tip section in FitBuddy provides users with concise, practical dietary advice to complement their workout plans. It emphasizes the importance of prioritizing protein intake in every meal, suggesting sources like chicken, fish, beans, or Greek yogurt. The tip explains how protein contributes to muscle building, satiety, and metabolic support—key factors for both fat loss and muscle gain. This section is designed to reinforce healthy eating habits that align with users' fitness goals.

Feedback Page:

Description: The Feedback Page in FitBuddy allows users to personalize and refine their fitness journey by submitting feedback on their current workout plans. Users are prompted to enter their unique User ID and provide specific feedback regarding the effectiveness, difficulty, or preferences related to their plan. Once submitted, this feedback is processed by the system to intelligently update and adjust the workout plan to better align with the user's goals, comfort, and progress. This dynamic feedback loop ensures that each fitness plan remains adaptive and tailored to the individual's evolving needs.

Once the plan is updated it comes with a confirmation message .

View all users page:

Description: The View All Users page in FitBuddy provides an overview of the entire user database, displaying detailed information for each registered user. This includes their personal details such as name, age, and fitness goals, along with their original workout plan generated at registration and the updated workout plan if any changes were made based on their submitted feedback. This feature enables a transparent view of how each user's plan has evolved over time, showcasing the adaptability of the FitBuddy system and helping administrators or developers monitor user progress and engagement effectively.

Conclusion:

The FitBuddy – AI Fitness Plan Generator is a comprehensive, user-centric web application that showcases the powerful synergy between artificial intelligence and health-tech solutions. Designed to assist individuals in achieving their fitness goals, FitBuddy uses Google’s Gemini 1.5 Pro and Gemini Flash models to generate personalized 7-day workout plans and daily nutrition or recovery tips based on user inputs such as age, weight, fitness goal, and preferred workout intensity.

The application is built using the FastAPI framework for backend logic and API handling, SQLAlchemy with SQLite for persistent data storage, and Jinja2 templating for a clean and responsive frontend. Upon submitting their details on the homepage, users receive a tailored workout plan that includes warm-ups, main workouts, cooldowns, and goal-aligned tips. What sets FitBuddy apart is its dynamic feedback feature—users can provide input on the effectiveness or difficulty of their current plan, and the system responds by intelligently updating the plan using generative AI, ensuring ongoing customization and engagement.

Additionally, the application includes an administrative view where all registered users and their respective plans (original and updated) are displayed in a structured format. This functionality supports transparency, monitoring, and further optimization, making FitBuddy suitable for both individual users and coaches or institutions managing multiple fitness profiles.

Overall, FitBuddy stands as a scalable and adaptable AI-powered solution that not only personalizes the fitness journey but also evolves with the user’s progress and preferences. It demonstrates how cutting-edge technology can be integrated into everyday wellness routines to promote healthy, consistent, and goal-oriented lifestyles.